Home
Main: https://www.haskell.org/haskellwiki/Haskell
Specifications (Haskell98): https://www.haskell.org/onlinereport/
Specifications (Haskell2010): https://www.haskell.org/onlinereport/haskell2010/

Libraries
Home: http://www.haskell.org/ghc/docs/latest/html/libraries/
Basic libraries: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/index.html
Prelude: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/Prelude.html

GHC
Home (User's guide): https://downloads.haskell.org/ghc/latest/docs/users_guide/index.html
Language extensions: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts.html

Haskell Wiki
Home: http://wiki.haskell.org/Haskell
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia
[Category] Language: https://wiki.haskell.org/Category:Language
Glossary: https://wiki.haskell.org/Category:Glossary

Classic Papers
Generalizing monads to arrows: http://www.cse.chalmers.se/~rjmh/citations/my_most_influential_papers.htm
Monadic constraints in programming (One of the first papers on monads?): https://homepages.inf.ed.ac.uk/wadler/topics/monads.html
Applicative programming with effects (First definition of Applicative): https://www.staff.city.ac.uk/~ross/papers/Applicative.html

Articles
vim plugins: http://www.reddit.com/r/haskell/comments/vomw0/awesome_haskell_plugins_for_vim
Real World Haskell: http://book.realworldhaskell.org/read/
What I wish I knew: https://github.com/sdiehl/wiwinwlh

==========

Tools

Glasgow Haskell Compiler (ghc)
Core Haskell compiler

Cabal
Home: https://www.haskell.org/cabal/
Package manager/builder

Haddock
Docs: https://haskell-haddock.readthedocs.io/en/latest/
Documentation builder

Haskell Stack
Home: https://docs.haskellstack.org/en/stable/
Tool for setting up haskell projects.

==========

Concepts

Wiki glossary: https://wiki.haskell.org/Category:Glossary

Haskell vs **OOP**
All Haskell **types** are like OOP template classes.
i.e. template are **first class** in Haskell
All Haskell **values** are like OOP class **objects**, instantiated by a **data constructor**.

Haskell	**OOP**
type constructor	template class
type = nullary type constructor	concrete class
type class	extending a class with methods e.g. C# extension methods
instance (of a type class)	a class on which the extension methods apply
data constructor	class constructor
value	class object
nullary data constructor as value e.g. True	Symbol
bottom	throwing an error
()	null
algebraic data type i.e. `data` declaration e.g. data MyType = MyType Int	Class declaration. But Haskell types are algebraic, instead of hierarchical i.e. Haskell types use composition, not inheritance
type synonym i.e. `type` declaration e.g. data MyInt = Int	type alias e.g. typedef
renamed type i.e. `newtype` declaration	?

Currying
Wiki: https://wiki.haskell.org/Currying
Currying is a **theoretical representation** of functions of **multiple** variables using only functions of **one** variable.
e.g.

multiple variables	**curried**
f : X1 x X2 x X3 → Y	f : X1 → (X2 → (X3 → Y))
	f(x1): X2 → (X3 → Y)
	f(x1)(x2): X3 → Y
(x1, x2, x3) ↦ ...	x1 ↦ (x2 ↦ (x3 ↦ ...)))

This significantly **simplifies** theory, since we can assume that **ALL** functions are **one variable**.

Functions of **multiple variables** are **derived** from nesting functions of multiple variables.

currying != partial application
Partial application: https://wiki.haskell.org/Partial_application
Currying is a **theoretical representation** of a function.
Partial application is a more **practical** concept
i.e. simply a function with some of its arguments already i.e. partially **applied**.
e.g. it is possible to have a partially applied function without currying

```
// e.g. JavaScript
const f = (x,y) => x*y;       // not curried
const g = x => f(x,2);        // g=partially applied f without
currying
```

currying + partial application in type constructors
In Haskell, **type constructors** are also curried.
Hence they can be partially applied.
i.e. partially applied type constructors are also treated as **types**.

```
-- e.g.
Either :: * -> * -> *         -- binary
Either Int :: * -> *          -- unary
Either a :: * -> *
Either Int Int :: *           -- nullary
Either a b :: *

-- e.g. usage: in class instances
instance Monad ((-> r)) where        -- unary type x ↦ r -> x  i.e.
fixed domain r
 fmap :: (.)                         -- e.g. Int -> x is a monad
over x
instance Monad (Either a) where      -- unary type x ↦ Either a x
is a monad.
 ...                                   -- e.g. Either Int x is a
monad over x
```

Strictness
Weak head normal form (WHNF)
Wiki - Strictness: http://wiki.haskell.org/Performance/Strictness
Wiki - WHNF: http://wiki.haskell.org/Weak_head_normal_form
Wiki - non-strict: http://wiki.haskell.org/Non-strict_semantics
Wikibooks: https://en.wikibooks.org/wiki/Haskell/Graph_reduction#Weak_Head_Normal_Form
????

Effect
Effectful (aka impure) logic (<=> pure logic)
(Algebraic) Effects system

Effect system (Diehl): https://www.stephendiehl.com/posts/exotic03.html
An **effectful** (aka **impure**) **computation** is a computation with a **side effect**.
An (algebraic) **effect system** is a way of representing effects in a pure system.
In Haskell, there are different ways of representing effects:
a) **monads**: e.g. IO monad

b) **free monads**: even more pure, since the **caller** decides which **interpretation** (aka **implementation**) to use.
May have **multiple** interpretations.
e.g. a mock interpretation for testing
e.g. different interpretation depending on platform (macOS, Linux, Windows, etc)
Introduction to free monads: https://serokell.io/blog/introduction-to-free-monads#using-free-monads

Higher-kinded type
What are higher kinded types: https://serokell.io/blog/kinds-and-hkts-in-haskell#what-are-higher-kinded-types-in-haskell%3F
A form of **nested polymorphism**
i.e. a polymorphic type with **polymorphic variables** (aka **higher-kind**) e.g. variables with kind (f :: Type -> Type)
e.g.

```
  -- NOT higher-kinded
  type Bool :: Type                                  Monomorphic type
  data Bool = True | False

  type Maybe :: Type -> Type                         (Parametric) Polymorphic type with monomorphic variable kind (a :: Type)
  data Maybe a = Just a | Nothing

  type Num :: Type -> Constraint                     (Adhoc) Polymorphic type with monomorphic variable kind (a :: Type)
  class Num a


  -- Higher-kinded
  type Compose ::                                    (Parametric) Polymorphic type with polymorphic variable kind (f, g :: Type -> Type)
    (Type -> Type) -> (Type -> Type) -> Type -> Type
        monomorphic variable kind (a :: Type)
  data Compose f g a = Compose (f (g a))

  type Functor :: (Type -> Type) -> Constraint       (Adhoc) polymorphic type with polymorphic variable kind (f :: Type -> Type)
  class Functor f
```

Kleisli category
Kleisli arrow
Kleisli category wiki: https://en.wikipedia.org/wiki/Kleisli_category
In Haskell, a **Kleisli arrow** is a function
 a -> m b Monad m
i.e. a function mapping to a monad
In particular, monad operation is a Kleisli
e.g.

```
 mx >>= k      k::a -> m b   i.e. Kleisli
```

Eta-expansion
Eta-reduction (point-free)
Eta-conversion
[HS Wiki] Eta conversion: https://wiki.haskell.org/Eta_conversion
[SE] What's the point of eta conversion: https://math.stackexchange.com/questions/65622/whats-the-point-of-eta-conversion-in-lambda-calculus
Eta conversion rule in **lambda calculus** is that two functions are equal if there values are equal

forall x. fx = gx iff f = g

Eta expansion is explicitly writing arguments.
Eta reduction (aka **point-free**) is
e.g.

f :: Int -> Int	== **reduce** ==>	f :: Int -> Int
f x = id x	<== **expand** ==	f = id

==========

Language specification

GHC language extensions
Standard Haskell is defined by the language spec: https://www.haskell.org/haskellwiki/Haskell
But the **standard compiler GHC** may implement a slightly different spec
i.e. **extensions** to the language.
Most are **extensions** i.e. additions to the spec.
e.g. Parallel list comprehension: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/parallel_list_comprehensions.html

```
{-# LANGUAGE ParallelListComp #-}            -- extension
[(x,y) | x <- [1,2,3] | y <- [3,4,5]
```

But also a few **deprecated** i.e. removed features.
e.g. No context in type declaration

```
-- allowed in language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2
- but disabled in GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/datatype_contexts.html
data Num a => MyType a = MyType a            -- ERROR in GHC

-- unless explicitly allowed
{-# LANGUAGE DatatypeContexts #-}            -- language option
data Num a => MyType a = MyType a            -- OK
```

Default extensions
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html
Some extensions are enabled by default
e.g. forall type signature

```
f :: forall a. a -> a
f = id
```

As of Haskell 2021, the default extensions are

The `GHC2021` language set comprises the following extensions:

- `BangPatterns`
- `BinaryLiterals`
- `ConstrainedClassMethods`
- `ConstraintKinds`
- `DeriveDataTypeable`
- `DeriveFoldable`
- `DeriveFunctor`
- `DeriveGeneric`
- `DeriveLift`
- `DeriveTraversable`
- `DoAndIfThenElse`
- `EmptyCase`
- `EmptyDataDecls`
- `EmptyDataDeriving`
- `ExistentialQuantification`
- `ExplicitForAll`
- `FieldSelectors`
- `FlexibleContexts`
- `FlexibleInstances`
- `ForeignFunctionInterface`
- `GADTSyntax`
- `GeneralisedNewtypeDeriving`
- `HexFloatLiterals`
- `ImplicitPrelude`
- `ImportQualifiedPost`
- `InstanceSigs`
- `KindSignatures`
- `MonomorphismRestriction`
- `MultiParamTypeClasses`
- `NamedFieldPuns`
- `NamedWildCards`
- `NumericUnderscores`
- `PatternGuards`
- `PolyKinds`
- `PostfixOperators`
- `RankNTypes`
- `RelaxedPolyRec`
- `ScopedTypeVariables`
- `StandaloneDeriving`
- `StandaloneKindSignatures`
- `StarIsType`
- `TraditionalRecordSyntax`
- `TupleSections`
- `TypeApplications`
- `TypeOperators`
- `TypeSynonymInstances`

Overview
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch1.html#x6-90001

A Haskell **program** consists of 4 levels:
- **modules**:
- **declarations**:
e.g. variable declaration
e.g. type declaration
- **expressions**: denotes a value
- **lexical structure**:

```
x=1*2                   declaration of variable "x" with expression
"1*2"
data MyType = A         declaration of type "MyType"
```

Expression
Value
Type
Every **expression** denotes a **value** of a **static type**
e.g. 1 value 1 of type Integer
A type can be **overloaded** i.e. extended with methods with a **type class**.

Names
6 types of names.
2 names denoting a **value**:
- **variable**
- **(data) constructor**

3 names denoting a **type**:
- **type variable**
- **type constructor**
- **type class**

Other:
- module

e.g.

```
x=1             -- variable "x" of expression "1"
True            -- value "True" defined by nullary data constructor
"True" i.e. nullary data constructor is also value
Just 1          -- value "Just 1" defined by unary data  "Just a"
Bool            -- type "Bool" defined by nullary type constructor
"Bool"
Maybe Int       -- type "Maybe Int" defined by unary type
constructor "Maybe a"
Maybe a         -- unary type constructor "Maybe a" with type
variable "a"
Num a => a      -- type class "Num" with type variable "a"


# e.g.
data Bool = True | false          -- nullary type constructor "Bool"
                                  -- 2 nullary data constructors
"True" "False"
data Maybe a = Just a | Nothing   -- unary type constructor "Maybe
a"
                                  -- unary data constructor "Just a"
and nullary data constructor "Nothing"
```

Lexical Structure
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch2.html#x7-140002

Identifiers
variable identifier
constructor identifier
Identifiers are case sensitive.
2 types of identifiers:
- **variable identifier**: begin with lower case or _

2 types of variable identifiers: **variable**, **type variable**
e.g. x=1 variable "x"
e.g. Num a => a type variable "a"

- **constructor identifier**: begin with upper case
3 types of constructor identifiers: **type constructor, type class, (data) constructor**
e.g. True (data) constructor
e.g. Integer type constructor
e.g. Num a => a type class "Num"

Expressions
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-220003

Functions
Types of functions
- **prefix function**: most functions
e.g. foldr
- **infix function** (aka **operator**)

Operator
An **operator** is an **infix function**

null-like values
Haskell has only 2 null-like values
- **bottom** ⊥ :equivalent to throwing an error in imperative languages
- **unit ()**: a non-error empty value
e.g. x = ()

Declarations
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-620004

User defined types
algebraic type "data"
type synonym "type"
renamed type "newtype"
type wiki: https://wiki.haskell.org/Type
newtype wiki: https://wiki.haskell.org/Newtype
Haskell has 3 ways of defining new types:

- **algebraic type**: the usual notion of defining a new type
i.e. analogous to defining a new **class** in OOP

- **type synonym**:
i.e. defining a **type alias** i.e. like **typedef** in OOP
Does NOT actually define a new type but just a different **label**.
Otherwise, exactly **identical** as the original type
i.e. compiler converts them to the same type
Primarily for **semantic** purposes
e.g. if a number represents a height, create an alias Height.

```
type Height = Int
x = 1 :: Height
x + 3                -- can use exactly like Int. 4
```

- **renamed type**:

Like a subcase of the algebraic type with just **one** type constructor and variable.

==========

OLD
1) **parametric** polymorphism: **type independent** polymorphism
Uniform implementation regardless of the underlying type
e.g. **existential type** variable in Haskell
Polymorphic types

```
data Maybe a = Just a | Nothing

data Either l r = Left l | Right r
```

Polymorphic functions

```
id :: forall a. a -> a
id a = a                            -- same for any a = Int, Bool,
Char, ...

fst :: forall a b. (a,b) -> a
fst (a,_) = a                       -- same for any a = Int, Bool,
Char, ...
```

2) **adhoc** polymorphism: **type dependent** polymorphism
Different implementation depending on the underlying type.
e.g. method overloading in OOP
e.g. **Type class** in Haskell

```
-- e.g. each instance defines its own succ :: a -> a implementation
class Enum a where
  succ :: a -> a
  pred :: a -> a

instance Enum Bool where
  succ False = True             -- succ for Bool

instance Enum Int where
  succ = (+1)                   -- succ for Int

instance Enum Char where
  succ = chr . (+1) . ord       -- succ for Char
```

Monomorphism

Opposite of polymorphism
i.e. any interface with **concrete types**.